Linguagem Irriga – APS Linguagens de Programação

Autor: Fernando Guerra Boni

---

1. Sobre a linguagem Irriga

A Irriga é uma linguagem de programação de domínio específico (DSL) pensada para descrever regras de irrigação de forma simples, usando uma sintaxe parecida com C.

A ideia é permitir que alguém escreva regras como “ler chuva”, “ligar bomba” e “abrir válvula” com estruturas familiares de programação (variáveis, `IF`, `WHILE`), sem precisar lidar diretamente com o assembly da máquina virtual.

A linguagem é compilada para uma VM existente, a **MicrowaveVM**, que funciona como máquina de baixo nível para executar o código gerado.

---

2. Visão geral da linguagem Irriga

A Irriga é uma linguagem imperativa simples, com:

- **Variáveis inteiras:**
  Atribuições como `x = 10` ou `x = x + 1`.

- **Condicionais:**
  `IF (condição) { ... } ELSE { ... }`

- **Laços:**
  `WHILE (condição) { ... }`

- **Expressões aritméticas:**
  Operadores `+`, `-`, `*`, `/` com inteiros.

- **Expressões booleanas:**
  Comparações `>`, `<`, `>=`, `<=`, `==`, `!=`
  e operadores lógicos `&&`, `||`.

A linguagem também define comandos específicos para sensores e atuadores ligados ao cenário de irrigação.

---

3. Sensores e atuadores na Irriga

A Irriga oferece comandos de alto nível para leitura de sensores e acionamento de atuadores:

- **Sensores (leitura):**
    - `LER(CHUVA) -> x`
    - `LER(UMIDADE, z) -> x`
    - `LER(CISTERNA) -> x`
    - `LER(RELOGIO) -> x`
    - `LER(SOL) -> x`
    - `LER(VAZAO, z) -> x`
- **Atuadores (ação):**
    - `VALVULA_ON(z)` / `VALVULA_OFF(z)`
    - `BOMBA_ON` / `BOMBA_OFF`
    - `DELAY(t)`

No protótipo da APS, essas leituras são mapeadas para valores internos no código gerado, o suficiente para exercitar as estruturas de decisão e repetição da linguagem.

---

4. Flex, Bison e integração com MicrowaveVM

O compilador da Irriga foi implementado usando ferramentas padrão:

- **Análise léxica (Flex) – `lexer.l`:**
    - Reconhece palavras-chave (`IF`, `WHILE`, `LER`, `BOMBA_ON`, `VALVULA_ON`, etc.), números (`NUMBER`), identificadores (`IDENT`) e operadores.
    - Aceita palavras-chave em maiúsculas e minúsculas (por exemplo, `VALVULA_ON` ou `valvula_on`).

- **Análise sintática + geração de código (Bison) – `parser.y`:**
    - Implementa a gramática da linguagem: programa, comandos, blocos, expressões, expressões booleanas.
    - Em cada regra, ações em C geram o código da MicrowaveVM com chamadas a `fprintf(codeout, ...)`.
- **Programa principal – `main.c`:**
    - Entrada: arquivo `.irriga` (código da linguagem Irriga) e nome do arquivo `.mwasm` de saída.
    - Chama `yyparse()` e grava o assembly da VM no arquivo `.mwasm`.
- **Máquina virtual alvo – MicrowaveVM:**
    - VM que simula um micro-ondas, com registradores e instruções como `SET`, `DECJZ`, `GOTO`, `PUSH`, `POP`, `PRINT`, `HALT`.

O pipeline de uso é:

```
 make
./irriga_comp programa.irriga programa.mwasm
python3 main.py programa.mwasm   # na pasta da MicrowaveVM
```

    -

---

5. Exemplo de programa Irriga

Exemplo de código em Irriga que decide se deve ligar a bomba ou não com base na chuva:

```
LER(CHUVA) -> x;

IF (x > 50) {
    BOMBA_OFF;
} ELSE {
    BOMBA_ON;
    VALVULA_ON(1);
    DELAY(5);
    VALVULA_OFF(1);
    BOMBA_OFF;
```

```
}
```

Resumo do comportamento:

- `LER(CHUVA) -> x` obtém um valor de chuva.
- Se `x > 50`, a bomba é desligada.
- Caso contrário, a bomba é ligada, a válvula da zona 1 é aberta por um tempo (`DELAY(5)`), e depois tudo é desligado.

O compilador traduz esse programa para o assembly da MicrowaveVM, usando labels e instruções como `SET`, `DECJZ` e `GOTO` para representar o `IF/ELSE`, o controle de fluxo e o atraso (`DELAY`).